Instructions

Build a data pipeline using custom functions to extract, transform, aggregate, and load e-commerce data.

1. Implement a function named `transform()` with one argument, taking `merged_df` as input, filling missing numerical values (using any method of your choice), adding a column `"Month"`, keeping the rows where the weekly sales are over $10,000 and drops the unnecessary columns. Ultimately, it should return a DataFrame and be stored as the `clean_data` variable.

2. Implement the function `avg_weekly_sales_per_month` with one argument (the cleaned data). This function will calculate the average monthly sales. For implementing this function you must select the "Month" and "Weekly_Sales" columns as they are the only ones needed for this analysis, then create a chain operation with `groupby()`, `agg()`, `reset_index()`, and `round()` functions, then group by the "Month" column and calculate the average monthly sales, then call `reset_index()` to start a new index order and finally round the results to two decimal places.

3. Create a function called `load()` that takes the cleaned and aggregated DataFrames, and their paths, and saves them as `clean_data.csv` and `agg_data.csv` respectively, without an index.

4. Lastly, define a `validation()` function that checks whether the two csv files from the `load()` exist in the current working directory.

Walmart is the biggest retail store in the United States. Just like others, they have been expanding their e-commerce part of the business. By the end of 2022, e-commerce represented a roaring $80 billion in sales, which is 13% of total sales of Walmart. One of the main factors that affects their sales is public holidays, like the Super Bowl, Labour Day, Thanksgiving, and Christmas.

In this project, you have been tasked with creating a data pipeline for the analysis of supply and demand around the holidays, along with conducting a preliminary analysis of the data. You will be working with two data sources: grocery sales and complementary data.

`grocery_sales`

- `"index"` - unique ID of the row
- `"Store_ID"` - the store number
- `"Date"` - the week of sales

- `"Weekly_Sales"` - sales for the given store

Also, you have the `extra_data.parquet` file that contains complementary data:

`extra_data.parquet`

- `"IsHoliday"` - Whether the week contains a public holiday - 1 if yes, 0 if no.
- `"Temperature"` - Temperature on the day of sale
- `"Fuel_Price"` - Cost of fuel in the region
- `"CPI"` – Prevailing consumer price index
- `"Unemployment"` - The prevailing unemployment rate
- `"MarkDown1"`, `"MarkDown2"`, `"MarkDown3"`, `"MarkDown4"` - number of promotional markdowns
- `"Dept"` - Department Number in each store
- `"Size"` - size of the store
- `"Type"` - type of the store (depends on `Size` column)

You will need to merge those files and perform some data manipulations. The transformed DataFrame can then be stored as the `clean_data` variable containing the following columns:

- `"Store_ID"`
- `"Month"`
- `"Dept"`
- `"IsHoliday"`
- `"Weekly_Sales"`
- `"CPI"`
- "`"Unemployment"`"

After merging and cleaning the data, you will have to analyze monthly sales of Walmart and store the results of your analysis as the `agg_data` variable that should look like:

Month	Weekly_Sales
1.0	33174.178494
2.0	34333.326579
...	...

Finally, you should save the `clean_data` and `agg_data` as the csv files.

In [3]:
```
import pandas as pd
import sqlite3
from sqlalchemy import create_engine
import os
```

In [4]:
```
# Load CSV into DataFrame
df = pd.read_csv('../data_raw/grocery_sales.csv')
```

```
# Create SQLite database
conn = sqlite3.connect("../data_raw/grocery_sales.db")
df.to_sql("grocery_sales", conn, if_exists="replace", index=False)
conn.close()
print("Data loaded into SQLite database successfully.")
```

```
Data loaded into SQLite database successfully.
```

In [5]:
```
query = """ SELECT * FROM grocery_sales; """
engine = create_engine("sqlite:///../data_raw/grocery_sales.db")
grocery_sales = pd.read_sql_query(query, engine)
grocery_sales
```

Out[5]:

	Unnamed: 0	index	Store_ID	Date	Dept	Weekly_Sales
0	0	0	1	2010-02-05	1	24924.50
1	1	1	1	2010-02-05	26	11737.12
2	2	2	1	2010-02-05	17	13223.76
3	3	3	1	2010-02-05	45	37.44
4	4	4	1	2010-02-05	28	1085.29
...	...	...	...	...	...	...
231517	231517	232414	24	2011-05-06	8	49471.07
231518	231518	232415	24	2011-05-06	50	1210.00
231519	231519	232416	24	2011-05-06	87	25893.32
231520	231520	232417	24	2011-05-06	85	1357.83
231521	231521	232418	24	2011-05-06	35	3648.91

231522 rows × 6 columns

In [7]:
```
# # Just use the CSV file directly
# grocery_sales = pd.read_csv('../data_raw/grocery_sales.csv')

# Extract function is already implemented for you
def extract(store_data, extra_data):
    extra_df = pd.read_parquet(extra_data)
    merged_df = store_data.merge(extra_df, on = "index")
    return merged_df

# Call the extract() function and store it as the "merged_df" variable
merged_df = extract(grocery_sales, "../data_raw/extra_data.parquet")
merged_df
```

Out[7]:

	Unnamed: 0	index	Store_ID	Date	Dept	Weekly_Sales	IsHoliday	Tempera
0	0	0	1	2010-02-05	1	24924.50	0	4
1	1	1	1	2010-02-05	26	11737.12	0	4
2	2	2	1	2010-02-05	17	13223.76	0	4
3	3	3	1	2010-02-05	45	37.44	0	4
4	4	4	1	2010-02-05	28	1085.29	0	4
...	...	...	...	...	...	...	...	
231517	231517	232414	24	2011-05-06	8	49471.07	0	5
231518	231518	232415	24	2011-05-06	50	1210.00	0	5
231519	231519	232416	24	2011-05-06	87	25893.32	0	5
231520	231520	232417	24	2011-05-06	85	1357.83	0	5
231521	231521	232418	24	2011-05-06	35	3648.91	0	5

231522 rows × 18 columns

In [8]:
```
# Create the transform() function with one parameter: "raw_data"
def transform(raw_data):
  # Fill NaNs using mean since we are dealing with numeric columns
  # Set inplace = True to do the replacing on the current DataFrame
    raw_data.fillna(
      {
          'CPI': raw_data['CPI'].mean(),
          'Weekly_Sales': raw_data['Weekly_Sales'].mean(),
          'Unemployment': raw_data['Unemployment'].mean(),
      }, inplace = True
    )
```

```
    # Define the type of the "Date" column and its format
    raw_data["Date"] = pd.to_datetime(raw_data["Date"], format = "%Y-%m-%d")
    # Extract the month value from the "Date" column to calculate monthly sa
    raw_data["Month"] = raw_data["Date"].dt.month

    # Filter the entire DataFrame using the "Weekly_Sales" column. Use .loc
    raw_data = raw_data.loc[raw_data["Weekly_Sales"] > 10000, :]

    # Drop unnecessary columns. Set axis = 1 to specify that the columns sho
    raw_data = raw_data.drop(["index", "Temperature", "Fuel_Price", "MarkDow
    return raw_data


# Call the transform() function and pass the merged DataFrame
clean_data = transform(merged_df)
clean_data
```

Out[8]:

	Unnamed: 0	Store_ID	Dept	Weekly_Sales	IsHoliday	CPI	Unemploym
0	0	1	1	24924.50	0	211.096358	8.106(
1	1	1	26	11737.12	0	211.096358	8.106(
2	2	1	17	13223.76	0	211.096358	8.106(
5	5	1	79	46729.77	0	211.096358	7.500(
6	6	1	55	21249.31	0	211.096358	7.500(
...	...	...	...	...	...	...	
231513	231513	24	40	45396.26	0	134.514367	8.212(
231515	231515	24	93	41295.84	0	134.514367	8.212(
231516	231516	24	9	24024.18	0	134.514367	8.212(
231517	231517	24	8	49471.07	0	134.514367	8.212(
231519	231519	24	87	25893.32	0	134.514367	8.212(

106231 rows × 8 columns

In [9]:

```
# Create the avg_weekly_sales_per_month function that takes in the cleaned c
def avg_weekly_sales_per_month(clean_data):
        # Select the "Month" and "Weekly_Sales" columns as they are the only
    holidays_sales = clean_data[["Month", "Weekly_Sales"]]
        # Create a chain operation with groupby(), agg(), reset_index(), and
    # Group by the "Month" column and calculate the average monthly sales
    # Call reset_index() to start a new index order
    # Round the results to two decimal places

    holidays_sales = (holidays_sales.groupby("Month")
    .agg(Avg_Sales = ("Weekly_Sales", "mean"))
    .reset_index().round(2))
    return holidays_sales
```

```
# Call the avg_weekly_sales_per_month() function and pass the cleaned DataFr
agg_data = avg_weekly_sales_per_month(clean_data)
agg_data
```

Out[9]:

	Month	Avg_Sales
0	1.0	33174.18
1	2.0	34333.33
2	3.0	33220.89
3	4.0	33392.37
4	5.0	33339.89
5	6.0	34582.47
6	7.0	33922.76
7	8.0	33644.79
8	9.0	33258.05
9	10.0	32736.99
10	11.0	36594.03
11	12.0	39238.80

In [11]:
```
# Create the load() function that takes in the cleaned DataFrame and the agg
def load(full_data, full_data_file_path, agg_data, agg_data_file_path):
        # Save both DataFrames as csv files. Set index = False to drop the i
    full_data.to_csv(full_data_file_path, index = False)
    agg_data.to_csv(agg_data_file_path, index = False)

# Call the load() function and pass the cleaned and aggregated DataFrames wi
load(clean_data, "../data_cleaned/clean_data.csv", agg_data, "../data_cleane
```

In [12]:
```
# Create the validation() function with one parameter: file_path - to check
def validation(file_path):
        # Use the "os" package to check whether a path exists
    file_exists = os.path.exists(file_path)
    # Raise an exception if the path doesn't exist, hence, if there is no fi
    if not file_exists:
        raise Exception(f"There is no file at the path {file_path}")


# Call the validation() function and pass first, the cleaned DataFrame path,
validation("../data_cleaned/clean_data.csv")
validation("../data_cleaned/agg_data.csv")
```

In [13]:
```
# jupyter nbconvert --to html walmart-retail-data-pipeline.ipynb
```